

Techniques de Simulation Programmation Avancée

Rapport de projet

Zié Mamourou T. COULIBALY

Charly CHAIDRON

Haris HAPPI

Sitraka RAMAHALEO

24 décembre 2023



1 Contexte

On considère le fleuve Coulangine traversant la ville d'Aussec. On souhaite savoir quelle est la probabilité que ce fleuve déborde et inonde la ville au cours de l'année 2024, selon qu'on construise ou non des murets autour de son lit. On modélise les pluies par un ensemble de dates où elles tombent (on considère ces pluies instantanées), et des intensités pour chacune de ces dates. À partir de ces données, aléatoires, un modèle nous permet de calculer la hauteur d'eau dans le fleuve à tout moment de l'année, et donc tester si la hauteur maximale dépasse un certain seuil correspondant au début d'une inondation.

1.1 Données du modèle

On modélise les dates des pluies par un processus de Poisson inhomogène, de fonction intensité définie par :

$$\lambda(t) = \lambda_0(1 + \alpha \sin(4\pi t))$$

Où λ_0 et α sont des constantes du modèle. Et on suppose en plus que $0 \leq t \leq 1$.

On modélise les intensités de chacune des pluies par des variables aléatoires indépendantes, et indépendantes des dates des pluies, toutes de loi P_I de densité f_I définie par :

$$f_I(x) = \frac{c_I}{m + |x - x_0|^\eta}$$

Où c_I est une constante de normalisation, et $m > 0$, $x_0 > 0$ et $\eta > 2$ sont des paramètres du modèle.

La hauteur d'eau $H(t)$ dans le fleuve Coulangine au temps t est définie comme suit :

$$H(t) = \sum_{i \geq 1} I_i \times r(t - T_i)$$

Où r est la fonction de résorption de l'eau de ce fleuve :

$$r(t) = 1_{\mathbb{R}^+}(t) \exp(-vt)$$

en notant v la vitesse de résorption.

On notera $H_{\max}$ la hauteur maximale de l'eau dans le fleuve Coulangine au cours de l'année 2024 :

$$H_{\max} = \max_{0 \leq t \leq 1} H(t)$$

Résultat attendu :

Le but de ce projet consistera à déterminer la probabilité que le maximum des hauteurs de pluie dépasse une certaine hauteur h_0 . On notera cette probabilité :

$$p^*(h_0) = P[H_{\max} > h_0]$$

Et on essayera de l'estimer par une méthode de Monte-Carlo.

Analyse du problème

Après lecture du problème, nous avons pensé à la *Théorie de la ruine* mais dans le sens inverse. Dans la *Théorie de la ruine* présentée dans le cours, on cherche à déterminer la probabilité d'être en dessous d'un seuil de capital. Ici, on cherche plutôt la probabilité d'être au dessus d'un certain seuil.

Illustration graphique

Cette illustration nous a permis de comprendre que le maximum aura forcément lieu à une date de pluie. Ce qui nous aidera beaucoup par la suite, notamment en gain d'efficacité au niveau du calcul des hauteurs.

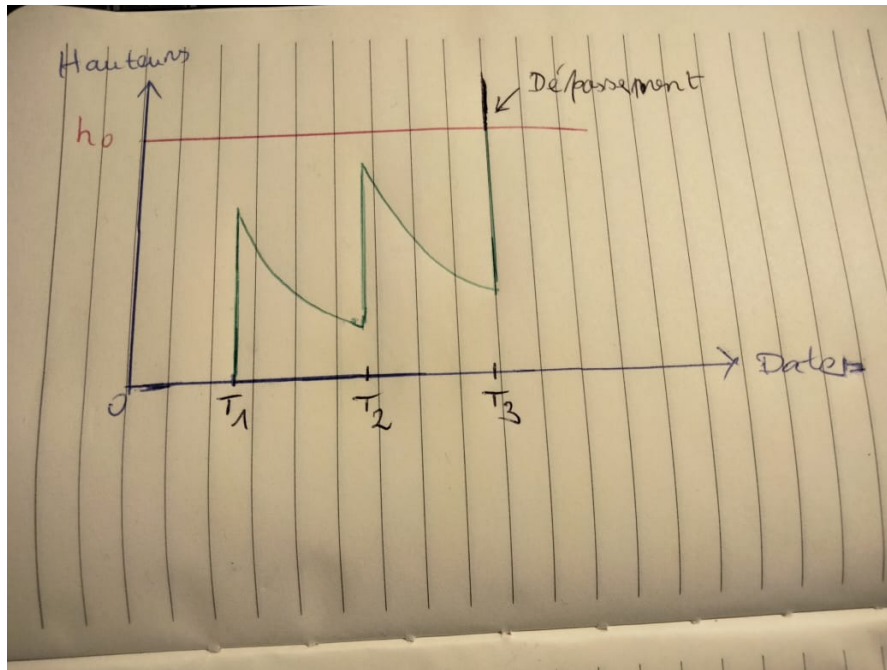


FIGURE 1 – Illustration d'un dépassement

2 Répartition du travail

Pour le projet, Zié COULIBALY s'est occupé de générer les intensités de pluie [rintensite](#) et d'optimiser le code.

Sitraka RAMAHALEO s'est occupée de générer les [temps de pluie](#).

Charly CHAIDRON s'est occupé de faire le code de [pétoile](#) et [petoile.petite](#).

Quant à Harris HAPPI, il s'est occupé de la [partie en C++](#).

Cependant, chacun de nous est intervenu sur chaque partie afin de relire et proposer des idées afin d'améliorer les codes et/ou de les accélérer.

3 Explication du code

Pour notre simulation, nous avons considéré les paramètres :

```
params <- list(m = 4, x0 = 13, eta = 4, lambda0 = 30, alpha = 0.5, v = 2, h0 = 300,
```

```
.
```

3.1 Simulation de la loi P_I

On note aussi f_1 , la fonction de densité de pluies, mais non normalisée i.e :

$$f_{1I}(x) = \frac{1}{m + |x - x_0|^\eta}$$

Pour simuler les intensités de pluie, nous avons utilisé un algorithme de rejet par rapport à une loi de Cauchy, comme présenté dans le cours (cf ci-dessous).

Algorithme de rejet (2)

Soit X une variable aléatoire dont la densité est proportionnelle à une fonction f .
On suppose qu'il existe une densité g telle que

- ▶ on sait simuler suivant la loi de densité g
- ▶ il existe c tel que $f(x) \leq cg(x)$ pour tout x .

Alors l'algorithme suivant simule la loi de X :

- 1 Répéter
- 2 Générer X suivant la loi de densité g
- 3 Générer Y suivant la loi $\mathcal{U}([0, c])$
- 4 jusqu'à $f(X) \geq g(X)Y$
- 5 Renvoyer X .

La majoration uniformément par une loi de Cauchy pose problème, car elle ne marche pas partout.

1^{er} cas :

Si $|x - x_0| > 1$, alors $|x - x_0|^\eta > |x - x_0|^2$ avec $\eta > 2$, donc $m + |x - x_0|^\eta > m + |x - x_0|^2$,
donc en passant à l'inverse on aura :

$$\frac{1}{m + |x - x_0|^\eta} \leq \frac{1}{m + |x - x_0|^2}$$

Dans ce cas, la majoration est bonne.

2^e cas :

Si $|x - x_0| < 1$, alors $|x - x_0|^\eta < |x - x_0|^2$ avec $\eta > 2$, donc $m + |x - x_0|^\eta < m + |x - x_0|^2$,
donc en passant à l'inverse on aura :

$$\frac{1}{m + |x - x_0|^\eta} \geq \frac{1}{m + |x - x_0|^2}$$

Et dans ce cas, la majoration n'est pas bonne.

Il a fallu donc chercher une constante C telle qu'on ait sur tout $\mathbb{R}^+$:

$$\frac{1}{m + |x - x_0|^\eta} \leq C \frac{1}{m + |x - x_0|^2} \quad (1)$$

.

L'inéquation (1) peut se réécrire

$$\frac{\frac{1}{m + |x - x_0|^\eta}}{\frac{1}{m + |x - x_0|^2}} \leq C$$

Ce qui est équivalent à :

$$\frac{m + |x - x_0|^2}{m + |x - x_0|^\eta} \leq C$$

C est une constante de majoration. Majorer une fonction qui s'écrit sous forme d'une fraction, revient à majorer le numérateur et à minorer le dénominateur.

On considère toujours les deux cas précédents.

1^{er} cas :

Si $|x - x_0| > 1$: On avait vu que la majoration par une densité de Cauchy (non normalisée) passait. Donc dans ce cas, le majorant C peut être pris égal à 1. 2^e

cas :

Si $|x - x_0| < 1$:

En majorant le numérateur, est $m + 1$, car : $m + |x - x_0|^2 \leq m + 1$.

Quant au dénominateur, un minorant, est 0, car : $m + |x - x_0|^\eta \geq 0$.

Donc pour ce cas, la constante C peut être prise égale à : $\frac{m + 1}{m}$.

Pour avoir un C qui majore partout, on doit choisir le maximum entre $\frac{m + 1}{m}$ et 1.

La constante $\frac{m + 1}{m}$ est plus grande que 1 car son numérateur est plus grand que son dénominateur. Donc on prendra

$$C = \frac{m + 1}{m}$$

Donc on aura sur tout $\mathbb{R}^+$, la majoration :

$$\frac{1}{m + |x - x_0|^\eta} \leq C \frac{1}{m + |x - x_0|^2}$$

En normalisant pour se ramener à une densité de Cauchy, on aura :

$$\frac{1}{m + |x - x_0|^\eta} \leq C \frac{\pi}{\sqrt{m}} \frac{\sqrt{m}}{\pi} \frac{1}{m + (x - x_0)^2}$$

Or une densité de Cauchy de paramètre x_0 et m est : $g(x) = \frac{\sqrt{m}}{\pi} \frac{1}{m + (x - x_0)^2}$.

On a donc :

$$f_{1I}(x) \leq C \cdot \frac{\pi}{\sqrt{m}} \cdot g(x) \quad (2)$$

.

Cependant en appliquant l'algorithme de rejet par rapport à une densité, on aura des valeurs négatives, on doit éviter cela car la densité des intensités des pluies est à valeurs dans $\mathbb{R}^+$.

Pour éviter cela, on va essayer de générer nous même des réalisations de Cauchy sur $\mathbb{R}^+$.

Pour se faire, on considère la méthode d'inversion de la fonction de répartition rappelée ci-dessous

Si U suit la loi uniforme sur $[0, 1]$, alors $G(U)$ a pour fonction de répartition F , de sorte que l'algorithme suivant simule la loi de fonction de répartition F :

```

1       $U \leftarrow$  générateur de loi  $\mathcal{U}([0, 1])$ 
2      Renvoyer  $G(U)$ 
```

Si on applique directement la méthode de l'image ci-dessus, on aura des réalisations négatives. Donc il faut chercher un réel u tel que $0 \leq u \leq 1$, et qui nous permettra de n'avoir que des réalisations positives.

La fonction quantile d'une densité de Cauchy de paramètre de position x_0 et d'échelle m est :

$$F^{-1}(x) = \sqrt{m} \tan\left(\pi\left(x - \frac{1}{2}\right)\right) + x_0$$

Où x est un élément de $[0, 1]$

Pour avoir des réalisations positives de cette densité de Cauchy, il faut :

$$F^{-1}(x) \geq 0$$

La résolution de cette inéquation mène à :

$$x \geq 0.5 + \frac{\arctan\left(-\frac{x_0}{\sqrt{m}}\right)}{\pi}$$

On pose donc

$$u = 0.5 + \frac{\arctan\left(-\frac{\text{params}\$x_0}{\sqrt{\text{params}\$m}}\right)}{\pi}$$

Et donc en générant des uniformes sur $[u, 1]$ on n'aura ainsi que des réalisations positives d'une loi de Cauchy, ce qui permet de gagner en efficacité et de réduire ainsi la zone de rejet.

Extrait de code

```

3 densite_non_normalisee <- function(x, params)
4 {
5   1/ (params$m + abs(x - params$x0)^params$eta)
6 }
7
8 u <- 0.5 + atan(-params$x0/sqrt(params$m))/pi
9
10 coef_inverse <- (params$m/(params$m+1))*(sqrt(params$m))/pi
11
12 rintensite <- function(n,params)
13 {
14   L <- c()
15   while(length(L)<n){
16
17     x <- runif(n-length(L),u,1)
18
19     x <- qcauchy(x,location = params$x0, scale=sqrt(params$m))
20
21     x <- x[x>0]
22
23     y <- runif(length(x), min=0, max=1)
24
25     D <- x[coef_inverse*densite_non_normalisee(x,params)>=y*dcauchy(x,params$x0,sqrt(params$m))]
26     L <- append(L,D)
27
28   }
29
30   return(L[1:n])
31 }
32

```

Dans le code, on a réécrit la majoration (2), en :

$$Coef_inverse * f_1 I(x) \leq g(x)$$

Où $Coeffinverse = \frac{1}{C \cdot \frac{\pi}{\sqrt{m}}} = \frac{m}{m+1} \frac{\sqrt{m}}{\pi}$ Comme ça au lieu de piocher des uniformes sur $[0, C \cdot \frac{\pi}{\sqrt{m}}]$, on les pioche plutôt sur $[0, 1]$, on s'est dits que cela serait plus facile à l'ordinateur (plus facile de choisir sur un intervalle petit que s'il était grand).

Au départ, on avait mis : si $x > 0$ renvoyer $\frac{1}{m + |x - x_0|^\eta}$ sinon renvoyer 0 (on utilise une fonction proportionnelle à la densité comme c'est marqué dans le cours). Or, tester à chaque fois dès le départ si on un élément est positif ou négatif peut être fastidieux, donc on l'avait enlevé, mais en l'enlevant on a constaté que les résultats de **petoile** et **petoile.petite** ramenait des valeurs un peu différentes. On a supposé que cela venait peut être du fait qu'il y avait des valeurs négatives, ce qui est impossible d'après le calcul qu'on a fait plus haut.

Pour corriger cela, on a donc rajouter la ligne 21 : $x \leftarrow x[x > 0]$, qui nous permet de faire un tri global et non élément par élément ce qui est un peu plus rapide.

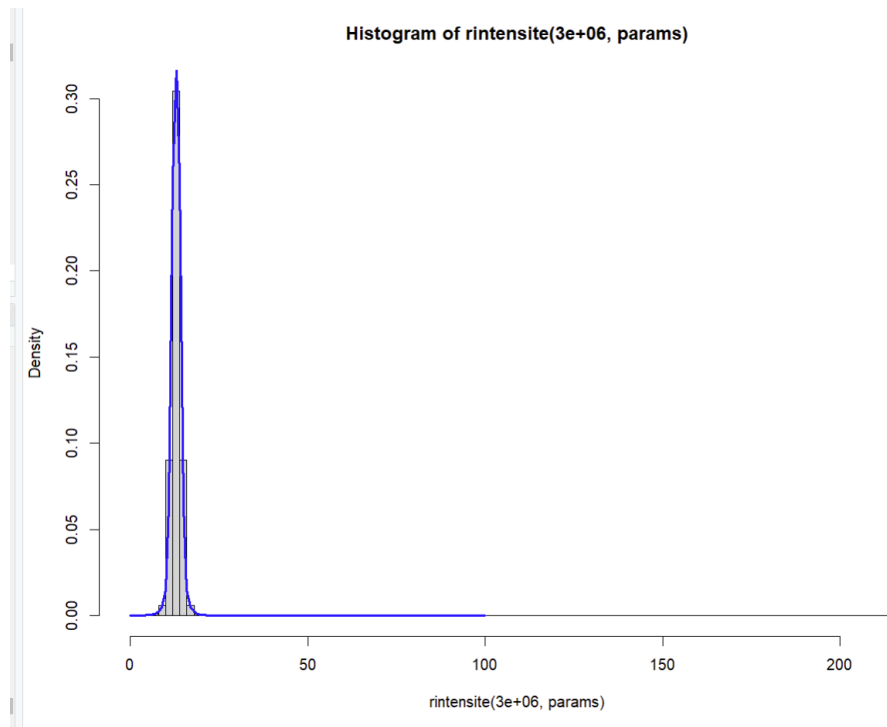
Pour vérifier si notre code était bon, on a calculé la moyenne et l'écart type empiriques et nous les avons comparés à ceux théoriques (via le calcul d'intégrale) et on a aussi fait un histogramme des valeurs simulées et fait passer la densité.

Histogramme

```

resultat=integrate(function(x) densite_non_normalisee(x, params), lower = 0, upper = Inf)$value
hist(rintensite(3000000,params),breaks=100,probability=TRUE)
curve(densite_non_normalisee(x,params)/resultat, 0,100,params=params,add=TRUE,lwd=3, col='blue')

```



dans notre code $1/\text{resultat}$ est la constante de normalisation.

4 Simulation des dates de pluie

Pour simuler un processus de Poisson inhomogène, nous simulons d'abord un processus de Poisson homogène sur $[0, T]$ (avec $T=1$). Ensuite, nous utilisons la méthode de simulation de rejet d'un processus de Poisson inhomogène, en prenant $\lambda_0(1 + \alpha)$ comme majorant de l'intensité λ . Dans notre programme, ce majorant est désigné par « intensité ».

```
lambda <- function(t){
  params$lambda0*(1+params$alpha*sin(4*pi*t))
}

proc_homo <- function(lambda,T){
  N <- rpois(1,lambda*T)
  L <- runif(N,0,T)
  return(sort(L))
}
```

```
rejet_poisson <- function(T,intensite){
  Tn <- proc_homo(intensite,T)
  U <- runif(length(Tn),0,1)
  X <- Tn[intensite * U <= lambda(Tn)]
  return(X)
}
```

En combinant les codes précédents, on a :

```
t_pluies <- function(T,intensite){
  N <- rpois(1,intensite*T)
  L <- runif(N,0,T)
  Tn <- sort(L)
  U <- runif(length(Tn),0,1)
  X <- Tn[intensite * U <= lambda(Tn)]
  return(X)
}
```

Après plusieurs simulations, on remarque qu'on obtient en moyenne λ_0 valeurs donc pour gagner du temps, on simule immédiatement un échantillon de taille λ_0 , cette fois-ci de loi exponentielle de paramètre $\lambda_0(1 + \alpha)$. Ensuite, au lieu de faire une somme petit à petit après chaque simulation d'une loi exponentielle, on utilise directement la fonction « cumsum » implémentée sur R qui nous permet de gagner en temps. Enfin, on ajuste l'échantillon i.e si le dernier temps de pluie est supérieur à 1 (puisqu'on veut un échantillon sur $[0, 1]$), on ne conserve que ceux qui sont inférieurs à 1, sinon on complète l'échantillon en générant des lois exponentielles de paramètre $\lambda_0(1 + \alpha)$ jusqu'à ce le dernier temps de pluie soit supérieur à 1.

Finalement, pour concours d'efficacité on a retenu ce code, dont la logique est expliquée au paragraphe précédent.

```

t_pluies <- function(T,intensite){
  X <- rexp(floor(params$lambda0), intensite)
  Y <- cumsum(X)
  if (Y[floor(params$lambda0)] > T){
    Tn <- Y[Y < T]
  }else{
    Tn <- Y
    TN <- Tn[length(Tn)]
    T0 <- TN
    X <- rexp(1, intensite)
    TN <- T0 + X
    while (TN < T){
      Tn <- c(Tn, TN)
      T0 <- TN
      X <- rexp(1, intensite)
      TN <- T0 + X
    }
  }
  U <- runif(length(Tn), 0, 1)
  X <- Tn[intensite * U <= lambda(Tn)]
  return(X)
}

```

5 Simulation des hauteurs de pluie

Un fois les méthodes de simulation de temps de pluies et d'intensité de pluies mises en place, nous cherchons à déterminer l'existence d'un débordement. La hauteur d'eau en un temps t étant donnée par la somme par pluie de l'intensité et de la fonction de résorption du temps écoulé depuis la pluie :

$$H(t) = \sum_{i \geq 1} I_i \times r(t - T_i)$$

On observe que la hauteur d'eau est continue par morceaux, entre deux pluies, strictement décroissante sur ces morceaux. Ainsi le maximum de la hauteur d'eau durant l'année, sera atteint lors d'une pluie, de même qu'un éventuel dépassement. Il nous suffit donc de calculer les hauteurs d'eau à chaque temps de pluie pour vérifier l'occurrence d'un dépassement.

Or, par récurrence, pour tout T_i temps de la i -ème pluie,

$$H(T_{i+1}) = H(T_i) \cdot \exp(-v \cdot (T_{i+1} - T_i)) + I_{i+1}$$

(Interpretation physique : La hauteur d'eau diminue d'une manière exponentielle entre le dates T_{n+1} et T_n , puis augmente d'une intensité : I_{n+1}).

De plus, nous ne sommes intéressés que par l'existence d'un débordement, pas par leur nombre, et pouvons donc arrêter notre simulation dès un débordement observé ce qui permet d'éviter de conserver des vecteurs des hauteurs et prendre le max et donc de gagner en temps d'exécution, on obtient ainsi le code suivant :

```

simul_h_pluies <- function(Tp, params) {
  I <- rintensite(length(Tp), params)
  H <- I[1]
  for (i in 1:(length(Tp) - 1))
  {
    if (H>params$h0){return(1)}
    H <- H * exp(params$v * (Tp[i] - Tp[i+1])) + I[i + 1]
  }
  if (H>params$h0){return(1)}
  return(0)
}

```

On observe que notre fonction ne renvoie, en fait, pas un tableau des hauteurs d'eau durant l'année, mais un binaire correspondant à l'existence d'un débordement Elle suit donc une loi de Bernoulli de probabilité la probabilité de débordement que l'on peut alors estimer par (qui par la loi des Grands Nombres, convergera vers la vraie probabilité) :

```

petoile<-function(n.simul,params)
{
  TH <- 0
  for(i in 1:n.simul)
  {
    TH<-TH+simul_h_pluies(t_pluies(1,params$lambda0 * (1 + params$alpha)),params)
  }
  result <- list(p = TH / n.simul, demi.largeur = 1.96 * sqrt((TH / n.simul) * (1 - TH / n.simul) / n.simul))
  return(result)
}

```

La demi-largeur associée est due à la normalité asymptotique d'une loi binomiale, il s'agit donc d'un intervalle de confiance asymptotique à 95%.

Pour *petoile.petite*, nous utilisons une méthode similaire, en s'inspirant de l'exemple sur *la théorie de la ruine*. Si ce n'est que, lors de notre appel de *tpluies()* dans *petoile()*, pour créer le tableau des temps à envoyer à *simulhpluies()*, nous augmentons le paramètre d'intensité (d'un facteur k que nous avons pris à 1,1% pour des raisons sur lesquelles nous reviendrons), ce qui a pour effet d'augmenter le nombre moyen de pluies dans l'année.

Ainsi *petoile* devient *petoile.petite* :

```

petoile.petite<-function(n.simul,params)
{
  TH <- 0
  for(i in 1:n.simul)
  {
    TH<-TH+simul_h_grande_pluies(t_pluies_frequentes(1,(1+k)*params$lambda0 * (1 + params$alpha)),params)
  }
  result <- list(p = TH / n.simul, demi.largeur = 1.96 * sqrt((TH / n.simul) * (1 - TH / n.simul) / n.simul))
  return(result)
}

```

Cette fois-ci, nous utilisons maintenant *tpluiesfrequentes*, il s'agit simplement d'un version de *tpluies*, modifiées pour utiliser $(1+k).\lambda(Tn)$ dans la méthode de rejet, et en général $(1+k).\lambda_0$:

```

k=0.011

t_pluies_frequentes <- function(T,intensite)
{
  X <- rexp(floor(params$lambda0), intensite)
  Y <- cumsum(X)
  if (Y[floor(params$lambda0)] > T)
  {
    Tn <- Y[Y < T]
  }
  else
  {
    Tn <- Y
    TN <- Tn[length(Tn)]
    T0 <- TN
    X <- rexp(1, intensite)
    TN <- T0 + X
    while (TN < T)
    {
      Tn <- c(Tn, TN)
      T0 <- TN
      X <- rexp(1, intensite)
      TN <- T0 + X
    }
  }
  U <- runif(length(Tn), 0, 1)
  X <- Tn[intensite * U <= (1+k)*lambda(Tn)]
  return(X)
}

```

Cependant, augmenter le nombre de pluies à un impact sur la probabilité de débordement, ce qui tombe plutôt bien vu que c'est l'objectif de la démarche : augmenter de manière maîtrisée la probabilité modélisée de débordement, pour pouvoir en avoir une estimation plus précise en moins de simulations, ce qui nous permet de gagner du temps machine, non pas pour un même nombre de simulations

(car pour chaque simulation on a plus de pluies) mais pour une même qualité de résultat.

Pour contrer une augmentation de k du nombre de pluies, il faut multiplier la probabilité finale par l'exponentielle de l'augmentation absolue et par le rapport des intensité initiale sur modifiée mis à la puissance du nouveau nombre de pluies :

$\exp(k \cdot \lambda_0) \cdot \left(\frac{1}{1+k}\right)^{\text{length}(Tp)}$ Pour cette démonstration, on a repris l'exemple de la théorie de la ruine mais ci, des processus de poisson inhomogène.

```

simul_h_grande_pluies <- function(Tp, params) {
  I <- rintensite(length(Tp), params)
  H <- I[1]
  for (i in 1:(length(Tp) - 1))
  {
    if (H>params$h0){return(exp(k*params$lambda0)*(1/(1+k))^length(Tp))}

    H <- H * exp(params$v * (Tp[i] - Tp[i+1])) + I[i+1]
  }
  if (H>params$h0){return(exp(k*params$lambda0)*(1/(1+k))^length(Tp))}
  return(0)
}

```

Nous aurions pu faire cette même modification au sein de `petoile.petite`, ça nous aurait évité du temps d'encodage, mais nous nous en rendons compte un peu tard.

Remarque : en moyenne, il y aurait $1.01 \cdot \lambda_0$ pluies par an, donc $\text{length}(Tp) \approx 1.01 \cdot \lambda_0$. Pour $\lambda_0 = \frac{365}{3}$ (il pleut environ tous les trois jours), on a $e^{0.01 \cdot \lambda_0} \left(\frac{1}{1.01}\right)^{1.01} \cdot \lambda_0 = 0.994$. Ce qui est faible, mais nous avons décidé de conserver cette valeur car elle renvoie de bons résultats au test présent sur Claroline.

Une modification de 10% de la moyenne du nombre de pluies demanderait de multiplier ce résultat par : $e^{0.1 \cdot \lambda_0} \left(\frac{1}{1.1}\right)^{1.1} \cdot \lambda_0 = 0.555$, ce qui signifie qu'une modification de 10% du nombre moyen de pluies aurait multiplié la probabilité de débordement par 1.8. Nous conseillons donc d'utiliser cette méthode pour des cas de probabilité véritablement faible, et avec prudence, pour ne pas se retrouver avec une probabilité estimée, avant correction, unitaire.

6 Partie en C++

Du point de vue algorithmique, les fonctions `petoile` et `petoile.c` sont totalement similaires. ELles ont toutes pour but de résoudre la même problématique qui est le calcul à l'aide de la méthode de Monte Carlo de la probabilité d'inondation de

la ville pour une hauteur de mur donnée. La différence entre ces deux fonctions algorithmiquement similaires se trouvent dans les langages utilisés. Comme son nom l'indique, la fonction `petoilec` est écrite en langage C++ et du R tandis que la fonction `petoile` est totalement rédigée en R. Le fait de déléguer une partie des calculs en C++ nous permet d'obtenir un résultat au moins fois plus rapidement. Pour cela, on a rédigé les sous-fonctions les plus demandantes en temps de `petoile` en C++ à savoir celle du calcul des temps de pluies et celle du calcul de Hauteur du niveau d'eau à chaque date de pluie.

```

NumericVector t_pluies_rcpp(double paramslambda0, double intensite, double T, double paramsalpha)
{
  RNGScope scope;
  NumericVector n(2);
  NumericVector X(floor(paramslambda0)), U(floor(paramslambda0 * 10));
  n = rpois(2, intensite * T);
  int lenL = 0;
  NumericVector Tn(2 * n[1]);
  Tn = runif(n[1], 0, T);
  U = runif(n[1], 0, T);
  for(int i = 0; i < n[1]; i++)
  {
    if(intensite * U[i] <= paramslambda0 * (1 + paramsalpha * sin(4 * M_PI * Tn[i])))
    {
      lenL = lenL + 1;
    }
  }
  NumericVector L(lenL);
  int lenA = 0;
  for(int i = 0; i < n[1]; i++)
  {
    if(intensite * U[i] <= paramslambda0 * (1 + paramsalpha * sin(4 * M_PI * Tn[i])))
    {
      L[lenA] = Tn[i];
      lenA = lenA + 1;
    }
  }
  std::sort(L.begin(), L.end());
  return(L);
}

```

Pour les hauteurs de pluie :

c'est juste la traduction de la fonction R de hauteur des pluies en C. Cette dernière utilise donc aussi la formule de récurrence calculée plus haut ce qui nous permet de faire des tests de déplacements au fur et à mesure sans jamais avoir à stocker toutes les valeurs différentes prises par la hauteur. La fonction est la suivante :

Ainsi cette fonction nous renvoie un booléen caractérisant l'évènement de dépassement de la hauteur du mur par le niveau d'eau c'est-à-dire l'évènement d'inondation.

Il nous restait juste à réunir ces deux fonctions de manière à pouvoir estimer notre probabilité en utilisant la méthode de Monte Carlo de manière identique à la fonction `petoile`.

Au final, la fonction `petoile.c` se retrouve au moins dix fois plus rapide au fur et à mesure qu'on augmente le nombre de simulations réalisées.


```

int hauteurRcpp(NumericVector a, NumericVector b, double paramshauteur, double paramsv) {
    double H = b[0];
    for (int i = 0; i < a.size() - 1; ++i) {
        if (H > paramshauteur) {
            return 1;
        }
        H = H * exp(paramsv * (a[i] - a[i + 1])) + b[i + 1];
    }

    if (H > paramshauteur) {
        return 1;
    }

    return 0;
}
}

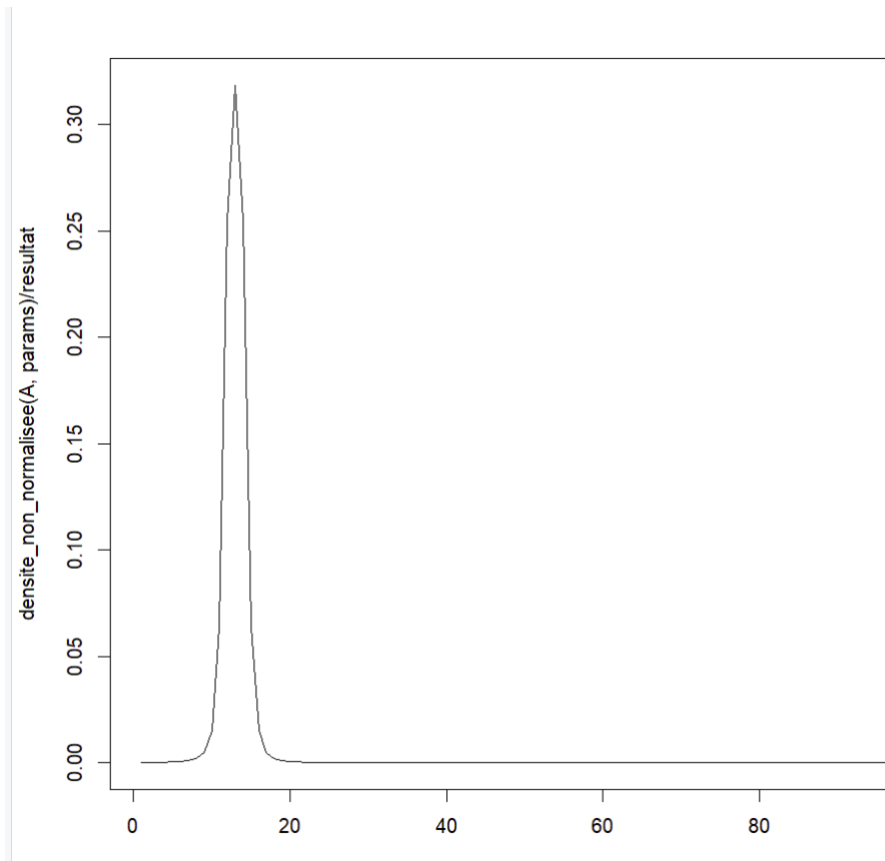
```

En terme d'évolution du modèle, on aurait aussi pu réalisé la fonction inverse à petoile qui en fonction d'une hauteur donnée nous donnerait la probabilité d'inondation, de ce fait on aurait plus qu'à rentrer le niveau à prendre de risque souhaité et on aurait la hauteur du mur à construire.

7 Influence des paramètres

7.1 Cas de x_0

Au vu des paramètres, on peut dire que x_0 sera le paramètre de position. En effet les simulations de [rintensite](#) sont autour de cette valeur. Quand on fait le graphique de la densité, cela se comprend. Le maximum de la fonction a lieu en $x = x_0$, donc c'est autour de cette zone que l'aire, la probabilité est importante. Pour notre cas : $x_0 = 13$, et on voit que c'est autour qu'il y 'a une zone non nulle et après, la fonctin se rapproche beaucoup de l'origine.



7.2 Cas de m Voir fin du document pour les images

Ici m est le paramètre d'échelle. Si m est grand alors la fonction de densité prendra très difficilement de grandes valeurs. Mais quand m est faible, alors on peut avoir de grandes valeurs.

Pour le cas $m = 2$, on voit que le maximum de la fonction est aux alentours de 0.7 tandis que pour $m = 8$, on est à peu près à 0.15

7.3 Cas de λ_0

Le paramètre λ_0 , est le nombre de pluies qu'on obtient en moyenne après simulation. Plus il est grand, et plus on a de pluies. Exemple : si $\lambda_0 = 30$, on aura en moyenne 30 pluies à l'année, sur plusieurs simulations.

7.4 Cas de η

Le paramètre η a une influence sur les intensités de pluie. Et aussi, sur les hauteurs par la suite. En effet, quand η est grand et plus les intensités sont fortes autour de $[-1 + x_0, 1 + x_0]$ (d'après la disjonction des cas faits plus hauts), alors que c'est dans cet intervalle que les réalisations de la densités sont plus probables. Et donc cela augmente légèrement la probabilité de dépassement du fleuve.

7.5 Cas de v

v désigne la vitesse de résorption, c'est la vitesse à laquelle les hauteurs de pluie diminuent (jusqu'à s'annuler en l'absence de nouvelle pluie). Plus v est grand, et plus la hauteur diminue rapidement, et plus est faible et plus les hauteurs diminuent lentement favorisant aussi un dépassement assez facile.

7.6 Cas de h_0

Ici, le seuil est h_0 . Afin de pouvoir espérer l'atteindre il faut que les autres paramètres x_0 et λ_0 et v soient ajustés.

Pour une grande hauteur, il faut un peu plus de pluie (donc λ_0 un peu grand) et aussi x_0 , car les intensités aident à déterminer les hauteurs de pluie, et les réalisations des intensités sont auour de la valeur x_0 .

8 Proposition d'évolution

Etant dans une école de gestion de risques en assurance, ce modèle peut faire penser à des débordements de vrais fleuves. Prenons le cas de **la seine**, qui est un fleuve qui coule dans le bassin parisien. Il peut paraître judicieux d'adapter ce modèle afin de calculer la probabilité que ce fleuve déborde de son lit, et d'en mesurer l'impact. En effet, un tel débordement aura plusieurs conséquences sur l'assurance automobile, habitation, car les voitures autour seront détruites et aussi les habitations.

Pour des cultivateurs dans les pays Africains, qui cultivent autour des fleuves, en cas de débordement, le fleuve peut détruire leur plantation. Et cela aura un impact également sur l'assurance agricole.

Pour rendre donc le modèle plus réaliste, nous proposons de poser le problème dans le cadre de la gestion de risques en assurances ainsi que les conséquences qui peuvent en découler, comme l'exemple donné avec la seine.

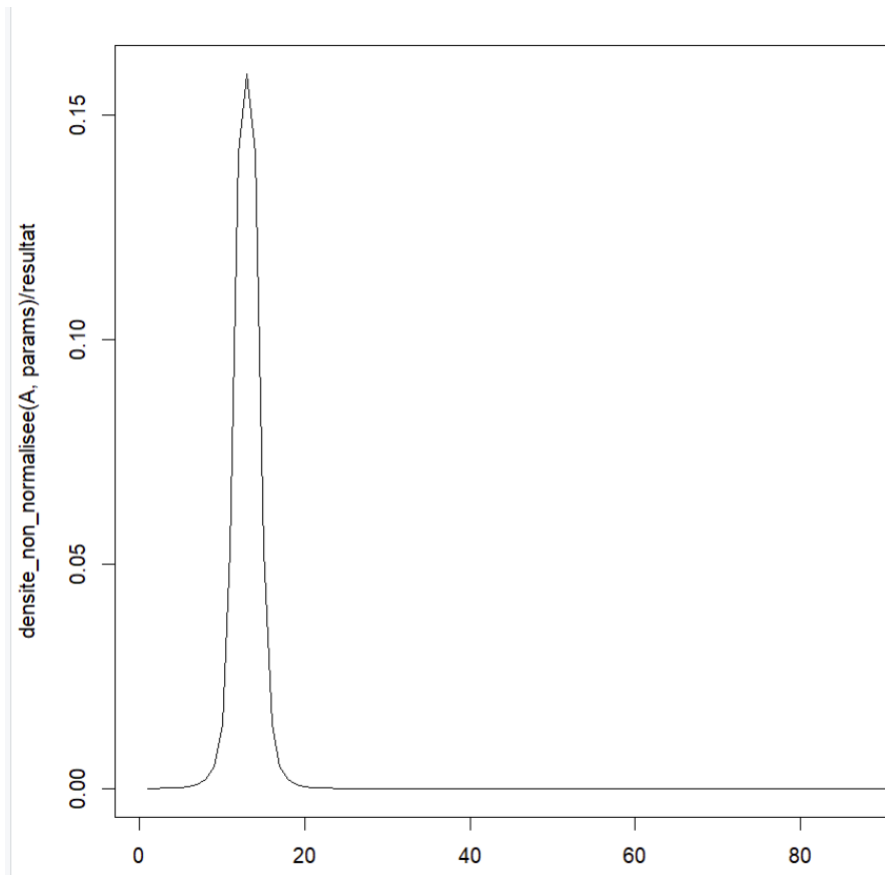


FIGURE 2 – Avec $m = 8$

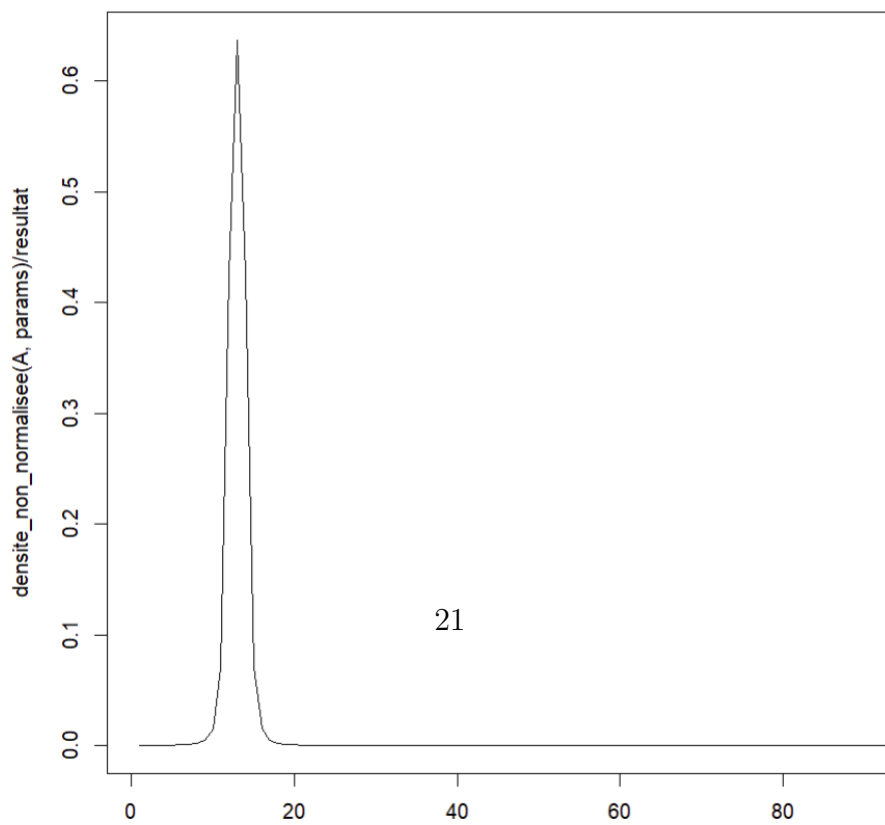


FIGURE 3 – Avec $m = 2$